

MODULE 2: SEARCH TECHNIQUES

Contents

1.	Comparative Study of Search Strategies	2
2	Uniform Search Strategies (Uninformed search)	2
2.1	Breadth-First Search (BFS).....	2
2.2	Depth-First Search (DFS)	3
2.3	Depth-Limited Search (DLS).....	4
2.4	Uniform Cost Search (UCS)	4
2.5	Bidirectional Search	5
2.6	Iterative Deepening Search (IDS)	6
2.7	Comparative Study	7
3	Heuristic Search Strategies.....	7
3.1	Best-First Search	8
3.2	Beam Search	9
3.3	A* Search.....	10
3.4	AO* Search (And-Or Graph Search).....	11
3.5	Memory-Bounded Heuristic Search	12
3.5.1	Hill-Climbing Search.....	13
3.5.2	Simulated Annealing Search.....	13
3.5.3	Local Beam Search Genetic Algorithm.....	14
3.6	Constraint Satisfaction Problems (CSPs).....	14
3.7	Local Search for CSPs.....	15
3.8	Comparative study.....	15
4	Adversarial Search	16
4.1	Games	16
4.2	Optimal Decisions & Strategies in Games.....	16

1. Comparative Study of Search Strategies

Criterion	Uniform Search	Heuristic Search	Adversarial Search
Information Used	No domain-specific knowledge	Uses domain-specific heuristics	Considers opponent's moves and strategies
Efficiency	Inefficient in large spaces (BFS, DFS)	More efficient if heuristics are good (A*)	Depends on tree size; alpha-beta improves speed
Optimality	BFS, UCS are optimal in specific conditions	A* is optimal with admissible heuristics	Minimax is optimal in zero-sum games
Completeness	BFS, UCS are complete	A*, Greedy Best-First can be complete (A*)	Minimax with alpha-beta pruning is complete
Memory Use	BFS is memory-heavy, DFS is memory-efficient	A* can be memory-intensive	Memory complexity depends on depth (DFS-based)
Examples of Use	Maze solving, pathfinding (UCS)	GPS navigation (A*), optimization (Sim. Anneal)	Chess, Go, other competitive games

2 Uniform Search Strategies (Uninformed search)

These strategies search the problem space without any additional information or heuristics about the goal location. They explore all possible paths systematically but do not guide the search based on any domain-specific information.

2.1 Breadth-First Search (BFS)

BFS is an uninformed search strategy that explores all nodes at the current depth level before moving on to the next level. It searches layer by layer, expanding all nodes at a given depth before going deeper.

- **Complete:** Yes, BFS will always find a solution if one exists.
- **Optimal:** Yes, if all edges or path costs are equal, BFS will find the shallowest (least cost) solution.
- **Time Complexity:** $O(b^d)$, where b is the branching factor (average number of successors per node) and d is the depth of the shallowest solution.
- **Space Complexity:** $O(b^d)$, BFS stores all nodes at the current level, so it uses a large amount of memory.

Advantages:

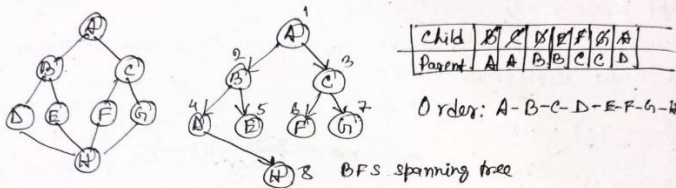
- Guarantees finding the optimal solution when all actions have the same cost.
- Complete and systematic, exploring the entire state space level by level.

Disadvantages:

- Memory-intensive due to the large number of nodes kept in memory, making it impractical for large depths.
- Slow for problems with high branching factors and deep solutions.

② BFS

① FIFO-BFS (this is default)



If edge costs = unit, then BFS-ST solves SSSP (Right?) problem in the most optimal way

TC of DFS & BFS depends on repr. of graph

(i) Adj. Matrix = $O(n^2)$

(ii) Adj. List = $O(n+e)$

SC = $O(n)$

2.2 Depth-First Search (DFS)

DFS is an uninformed search strategy that explores as far as possible along a branch before backtracking. It delves deep into the search tree first and only explores alternative branches when it reaches a dead end (leaf node) or the goal.

- **Optimal:** No, DFS does not guarantee finding the least cost or shallowest solution.
- **Complete:** No, DFS may fail to find a solution if it goes down an infinite path or gets stuck in a loop (unless cycles are handled or the state space is finite).
- **Time Complexity:** $O(b^m)$, where m is the maximum depth of the search tree.
- **Space Complexity:** $O(bm)$, which is much more memory-efficient than BFS as DFS only stores nodes on the current path.

Advantages:

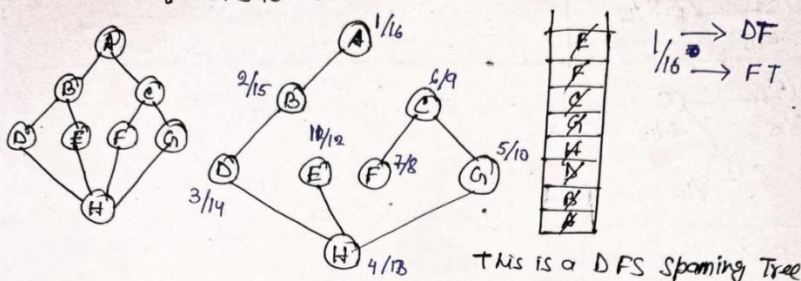
- Memory-efficient compared to BFS.
- Can find solutions quickly if they exist in deep branches.

Disadvantages:

- Not guaranteed to find the shallowest or optimal solution.
- Can get stuck in infinite loops unless precautions like cycle checking are implemented.

① DFS in undirected connected graph

Condition for DFS to come to halt is Stack becomes empty



Eg: which of the following orders are valid

- (a) A, B, E, H, D, F, C, G ✓
- (b) A, C, F, E, H, G, B, D ✗
- (c) H, D, B, E, A, C, F, G ✓
- (d) H, F, C, A, D, F, G ✗
- (e) G, C, F, H, E, B, D, A ✓

2.3 Depth-Limited Search (DLS)

DLS is a variation of DFS that imposes a limit on the depth to which the algorithm can search. It explores all nodes up to a predetermined depth and stops once it reaches this limit.

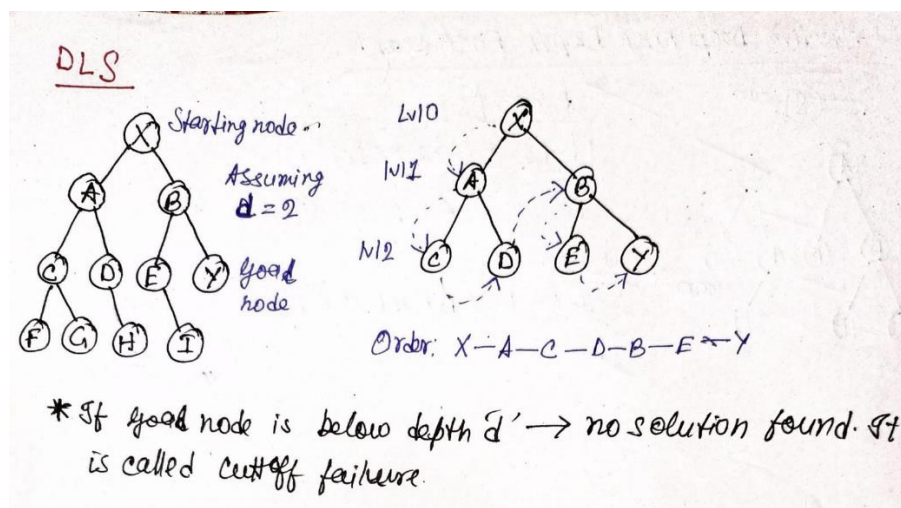
- **Complete:** Yes, if the depth limit l is greater than or equal to the solution depth.
- **Optimal:** No, like DFS, it is not guaranteed to find the shallowest or least-cost solution.
- **Time Complexity:** $O(b^l)$, where l is the depth limit.
- **Space Complexity:** $O(bl)$, making it more memory-efficient than BFS.

Advantages:

- Prevents DFS from going down infinite paths or getting stuck in cycles.
- More memory-efficient and suitable for deep search trees where the solution is expected within a certain depth.

Disadvantages:

- Requires a good estimate of the depth limit to avoid missing solutions.
- If the solution is deeper than the limit, it will not be found.
- Not optimal since it does not guarantee the least-cost solution.



2.4 Uniform Cost Search (UCS)

Uniform Cost Search (UCS) is a search algorithm that expands the node with the lowest cumulative cost from the start node, instead of prioritizing the depth of the node or using a heuristic. UCS is essentially a variant of Dijkstra's algorithm and is used to find the least-cost path in a weighted graph where all edge costs may not be uniform.

Properties of Uniform Cost Search

- **Completeness:** UCS is complete, meaning it is guaranteed to find a solution if one exists.
- **Optimality:** UCS is optimal, meaning it finds the least-cost path to the goal.
- **Time Complexity:** The time complexity of UCS is $O(b^{\lceil C^*/\epsilon \rceil})$, where:
 - B is the branching factor (the number of successors each node has).
 - C^* is the cost of the optimal solution.
 - ϵ is the minimum cost of any action (positive).

This means that UCS can take a long time if the solution involves many small-cost actions or if there are many nodes with similar costs.

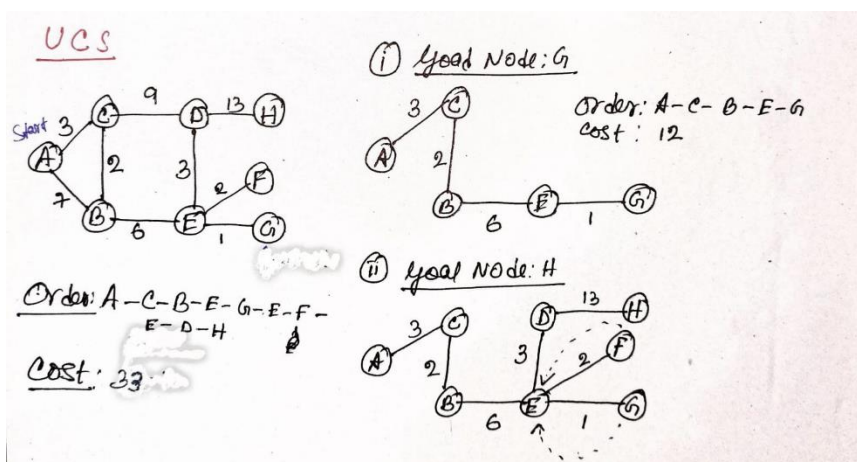
- **Space Complexity:** The space complexity is the same as time complexity, $O(b^{\lceil C^*/\epsilon \rceil})$, as UCS stores all the explored nodes in memory.

Advantages of Uniform Cost Search

- **Guarantees Optimality:** UCS guarantees to find the least-cost path to the goal, making it ideal for applications where finding the optimal solution is critical.
- **Applicable to Weighted Graphs:** Unlike simpler algorithms like breadth-first search (BFS), UCS works with weighted graphs where the cost of moving between nodes is not uniform.
- **No Heuristics Required:** UCS does not rely on heuristics, making it more straightforward to implement in situations where heuristic information is unavailable or unreliable.

Disadvantages of Uniform Cost Search

- **Time-Consuming:** UCS can be slow because it expands nodes in increasing cost order, which may lead to the exploration of many unnecessary nodes before reaching the goal.
- **High Memory Usage:** UCS stores all nodes in memory, including those with very low or negligible chances of contributing to the final solution.
- **Sensitive to Small Costs:** If the minimum edge cost ϵ is very small, UCS can take a long time to terminate, as it might explore many small-cost nodes before finding the goal.



2.5 Bidirectional Search

Bidirectional search is an uninformed search strategy that simultaneously searches forward from the initial state and backward from the goal, stopping when the two searches meet in the middle. Any search technique can be used here.

Properties:

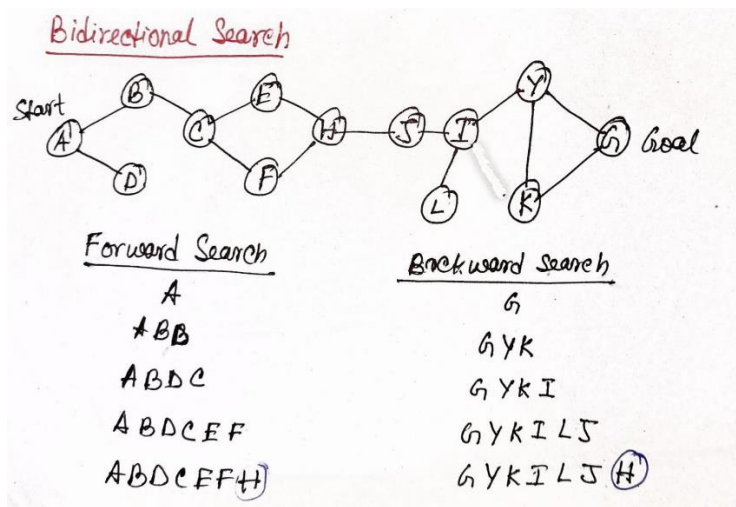
- **Complete:** Yes, if a solution exists, bidirectional search will find it.
- **Optimal:** Yes, if both searches use BFS and the path costs are uniform.
- **Time Complexity:** $O(b^{d/2})$, because each search covers half the depth, significantly reducing the time required compared to BFS or DFS.
- **Space Complexity:** $O(b^{d/2})$, making it more memory-efficient than BFS.

Advantages:

- Drastically reduces the time and memory needed by cutting the search space in half.
- Works well for large state spaces where both the start and goal are known.

Disadvantages:

- Requires a clear goal state, which might not always be feasible.
- Implementing a backward search can be challenging for some problem types.
- Memory requirements are still significant, though better than BFS.



2.6 Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS) is a search strategy that combines the space efficiency of **Depth-First Search (DFS)** with the completeness and optimality of **Breadth-First Search (BFS)**. IDS repeatedly applies DFS with increasing depth limits, gradually expanding the search boundary until the solution is found.

Properties of Iterative Deepening Search

- **Completeness:** IDS is **complete**, meaning that if a solution exists, it is guaranteed to find it.
- **Optimality:** IDS is **optimal** when the cost of all steps (actions) is equal.
- **Time Complexity:** For a branching factor b and depth d , the time complexity is $O(b^d)$.
- **Space Complexity:** IDS uses **very little memory**, with a space complexity of $O(d)$, where d is the depth of the search.

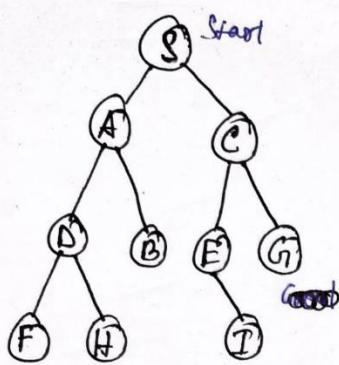
Advantages of Iterative Deepening Search

- **Memory Efficiency:** IDS uses the memory efficiency of DFS.
- **Completeness and Optimality:** IDS gives the **completeness** and **optimality** of BFS.
- **Simple Implementation:** IDS is relatively easy to implement.
- **Handles Infinite Depths:** IDS can handle search spaces that have infinite depth.

Disadvantages of Iterative Deepening Search

- **Repetition of Work:** A common criticism of IDS is that it repeatedly expands nodes at shallower depths.
- **Not Suitable for High-Cost Problems:** IDS works well when all edges or steps have equal cost.

Iterative Deepening Depth First Search



Goal = G
 $d=0$: [S]
 $d=1$: [S → A → C]
 $d=2$: [S A D B C E G]
Goal = I
 $d=3$: [S A B F H B C E I]

2.7 Comparative Study

Criterion	BFS	DFS	DLS	Bidirectional
Search Approach	Explores level by level (breadth-wise)	Explores as deep as possible (depth-wise)	Depth-first with a limit	Simultaneous forward and backward search
Completeness	Yes	No, unless space is finite	Yes, if the limit is sufficiently deep	Yes, if the two searches meet
Optimality	Yes, if all steps have equal cost	No	No	Yes, if using BFS with uniform costs
Time Complexity	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^{d/2})$
Space Complexity	$O(b^d)$	$O(b^m)$	$O(b^l)$	$O(b^{d/2})$
Memory Usage	High	Low	Low	Moderate, but less than BFS
Depth Limitation	No limit	No limit	Yes, imposed limit	No depth limit
Best for	Shallow solutions with equal costs	Deep solutions with minimal memory use	Deep searches with known depth limit	Large state spaces with known goals
Examples	Shortest path in a maze	Puzzle-solving with deep solutions	Problems with known depth constraints	Shortest path search with clear goals

3 Heuristic Search Strategies

Heuristic search algorithms use problem-specific knowledge (heuristics) to guide the search towards the goal more efficiently than uniform strategies.

- They are goal-directed (info about the goal state is present)

- Aiming to reduce the search space significantly by estimating how close a node is to the goal.
- Finds optimal solution to reach goal state using a heuristic function $F(n) = G(n) + H(n)$ [$G(n)$: Path cost, $H(n)$ Heuristic value]

Type of Heuristic

1. **Admissible Heuristics:** A heuristic is **admissible** if it **never overestimates** the true cost of reaching the goal from any given node. If $h(n)$ be the heuristic function and $h^*(n)$ be the true cost from node n to the goal. The heuristic is admissible if: $h(n) \leq h^*(n)$.
2. **Non-Admissible Heuristics:** A heuristic is **non-admissible** if it **can overestimate** the true cost of reaching the goal. If $h(n)$ be the heuristic function and $h^*(n)$ be the true cost from node n to the goal. The heuristic is admissible if: $h(n) > h^*(n)$.

Criteria	Admissible Heuristics	Non-Admissible Heuristics
Definition	Never overestimates the cost to reach the goal.	Can overestimate the cost to reach the goal.
Optimality	Guarantees finding the optimal solution (e.g., with A*).	May not find the optimal solution.
Performance	Often slower due to exploring all equally promising nodes.	Can be faster but risks missing the optimal solution.
Examples	Straight-line distance (Euclidean distance), Manhattan distance.	Overestimated distance, inaccurate heuristics.
Search Behaviour	Thorough, explores all possible optimal paths.	Can quickly explore paths but may lead to suboptimal solutions.

3.1 Best-First Search

Greedy Best-First Search is an informed search strategy that expands the node closest to the goal, according to a heuristic function $h(n)$ which estimates the cost to reach the goal from a given node. It "greedily" picks the node that appears to be closest to the goal based on this heuristic.

Properties:

- **Complete:** No, it can get stuck in loops or local optima if the heuristic is poorly defined.
- **Optimal:** No, because it does not account for the cost of reaching a node, only the estimated cost to the goal.
- **Uses priority Queue:** to store cost of nodes
- **Time/Space Complexity:** Depends heavily on the quality of the heuristic, but generally $O(b^m)$ where b is the branching factor and m is the maximum depth.

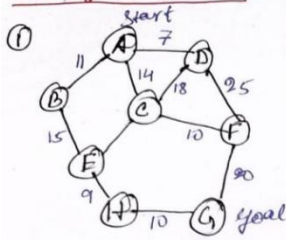
Advantages:

- Can be faster than uninformed search algorithms when a good heuristic is available.
- Uses a heuristic to focus the search on promising paths.

Disadvantages:

- Not guaranteed to find the optimal solution.
- Can perform poorly in large or complex search spaces if the heuristic is inaccurate.

Best first Search

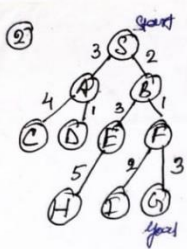


Straight line distances (heuristics)

$A \rightarrow G = 40$	$G \rightarrow G = 0$
$B \rightarrow G = 32$	$H \rightarrow G = 10$
$C \rightarrow G = 25$	
$D \rightarrow G = 35$	
$E \rightarrow G = 19$	
$F \rightarrow G = 17$	

open	closed	Description
[A]	[]	A is read and other nodes adjacent to A are put in open in ascending order of open heuristics.
[C, B, D] 25 32 35	[A]	C is read & same
[F, E, B, D]	[A, C]	F is read & same.
[G, E, B, D]	[A, C, F]	G and same.
[H, E, B, D]	[A, C, F, G]	Stop as we got goal G.

Path: $A \xrightarrow{14} C \xrightarrow{10} F \xrightarrow{20} G$ Cost = 44



$A \rightarrow 12$	Open	closed
$B \rightarrow 4$	[S]	[S]
$C \rightarrow 3$	[B, A]	[S]
$D \rightarrow 3$	[F, E, A]	[S, B]
$E \rightarrow 8$	[G, I, E, A]	[S, B, F]
$F \rightarrow 2$	[I, E, A]	[S, B, F, G]
$G \rightarrow 0$		
$H \rightarrow 4$		
$I \rightarrow 19$		
$S \rightarrow 13$		

Path: $S \rightarrow B \rightarrow F \rightarrow G$ Cost: 6

3.2 Beam Search

Beam Search is a **heuristic search algorithm** that explores a graph by expanding the most promising nodes, but it restricts the memory and time usage by limiting the number of nodes it keeps at each level. Beam search works like **Breadth-First Search (BFS)**, but instead of exploring all successors of a node, it keeps only a fixed number (called the beam width) of the best candidates based on a heuristic function.

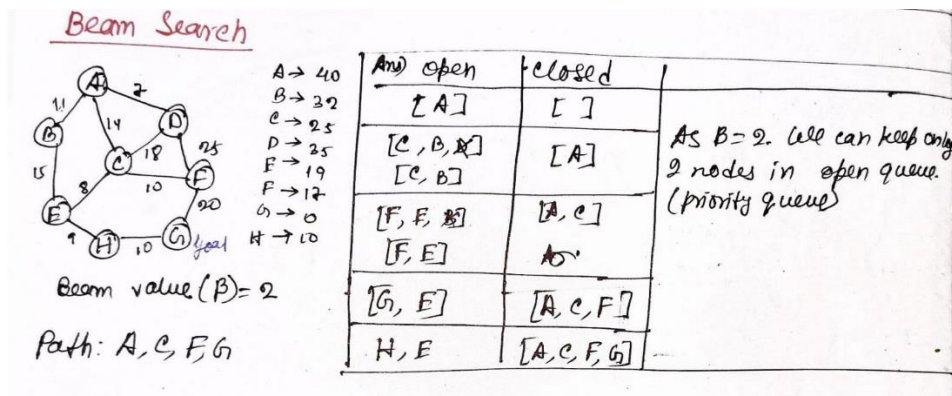
- **Completeness:** Beam search is **not complete**, as it can discard paths that may lead to the goal due to the beam width restriction.
- **Optimality:** Beam search is **not optimal** because it may discard better paths early in the search due to the beam width constraint.

Advantages:

- **Memory Efficient:** The beam width limits the number of nodes in memory, making it more memory-efficient compared to BFS.
- **Speed:** By discarding fewer promising paths early, it often runs faster than exhaustive search algorithms.

Disadvantages:

- **Risk of Missing Solutions:** Beam search might discard promising paths and fail to find the solution if the beam width is too narrow.
- **Non-Optimality:** The algorithm does not guarantee the optimal solution due to the discarding of potential paths.



3.3 A* Search

A* is an informed search algorithm that combines both the path cost $g(n)$ and the heuristic estimate $h(n)$ to select the node with the lowest total cost, using the evaluation function $f(n)=g(n)+h(n)$.

- $g(n)$: Cost of the path from the start node to the current node.
- $h(n)$: Estimated cost from the current node to the goal (heuristic value).

Properties:

- **Complete:** Yes, if the heuristic is admissible (i.e., it never overestimates the true cost).
- **Optimal:** Yes, if the heuristic is admissible and consistent (monotonic).
- **Time/Space Complexity:** $O(b^d)$, but can be reduced with a good heuristic. Memory usage is often the limiting factor because it stores all generated nodes.

Advantages:

- Guaranteed to find the optimal solution when using an admissible heuristic.
- Combines the benefits of path cost and heuristic information.

Disadvantages:

- Can be memory-intensive, especially in large search spaces.
- Its performance depends heavily on the quality of the heuristic function.

A* Search

①

$S(7)$
 $A(6)$
 $B(2)$
 $C(1)$
 $D(0)$

$S \rightarrow A = h + g = 6 + 1 = 7$
 $S \rightarrow B = h + g = 4 + 2 = 6$ } $S \rightarrow B$ chosen

$S \rightarrow B \rightarrow C = (4 + 2) + 1 = 7$
 $S \rightarrow B \rightarrow C \rightarrow D = (4 + 2 + 3) + 0 = 9$

$S \rightarrow A \rightarrow B = (1 + 2) + 2 = 5$
 $S \rightarrow A \rightarrow C = (1 + 5) + 1 = 7$
 $S \rightarrow A \rightarrow D = (1 + 12) + 0 = 13$ } for $S \rightarrow A$ we see

$S \rightarrow A \rightarrow B \rightarrow C = (1 + 2 + 2) + 1 = 6$
 $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D = (1 + 2 + 2 + 3) + 1 = 8$ } for $S \rightarrow A \rightarrow B$

$S \rightarrow A \rightarrow C \rightarrow D = (1 + 5 + 3) + 0 = 9$ } for $S \rightarrow A \rightarrow C$

Among the following 4 solutions we choose the least one.
 So $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$. Cost = 8

②

$S(5)$
 $A(3)$
 $B(4)$
 $C(2)$
 $D(6)$
 $G(0)$

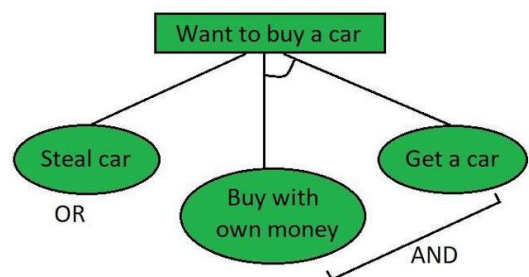
$S \rightarrow A = 1 + 3 = 4$ ✓
 $S \rightarrow B = 2 + 4 = 6$
 $S \rightarrow A \rightarrow B = 3 + 4 = 7$
 $S \rightarrow A \rightarrow C = 2 + 2 = 4$ ✓
 $S \rightarrow A \rightarrow C \rightarrow D = 5 + 6 = 11$ ✓
 $S \rightarrow A \rightarrow C \rightarrow G = 6 + 0 = 6$

$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 7 + 0 = 7$
 $S \rightarrow A \rightarrow B \rightarrow D = 8 + 6 = 14$
 $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 10$

Ans: $S \rightarrow A \rightarrow C \rightarrow G$
 Cost = 6

3.4 AO* Search (And-Or Graph Search)

AO* (And-Or) Search is a **graph-based search algorithm** used for solving problems with **AND/OR graphs**, where some parts of the problem require satisfying multiple conditions (AND) and others may only need one of several possibilities (OR). AO* is designed to handle problems where subproblems may have dependencies, and the solution may involve a combination of different paths.



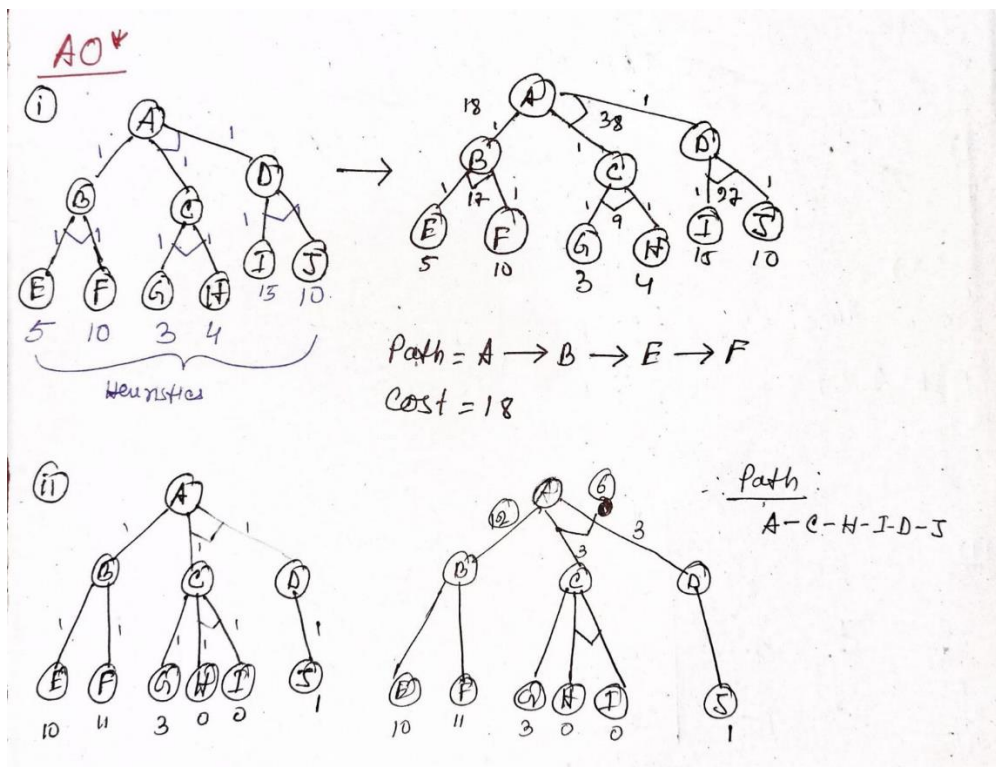
- **Completeness:** AO* is **complete** and can find a solution if one exists.
- **Optimality:** It is **optimal** when using admissible heuristics, as it will find the least-cost solution in an AND/OR graph.

Advantages:

- **Handles AND/OR Dependencies:** It can efficiently handle problems with dependencies between subproblems (AND nodes).
- **Optimal:** With an admissible heuristic, AO* is guaranteed to find the best solution.

Disadvantages:

- **Complex to Implement:** AND/OR graphs are more complex than traditional OR-only graphs, and implementing AO* requires careful handling of the dependencies.
- **Memory-Intensive:** Keeping track of multiple paths and their dependencies can be memory-intensive.



3.5 Memory-Bounded Heuristic Search

Memory-bounded heuristic search algorithms are designed to reduce the memory requirements of algorithms like A*. They provide solutions in memory-constrained environments.

Types of Memory-Bounded Heuristic Searches:

1. **Iterative Deepening A* (IDA*):** A memory-efficient variant of A* that uses depth-first search combined with an iterative deepening approach.
 - **How it works:** Performs a series of depth-first searches, each with a limit on the total cost $f(n)$. The cost limit is increased iteratively.
 - **Memory Usage:** Significantly lower than A* since it does not store all nodes.
 - **Time Complexity:** Still exponential, but more memory-efficient.
2. **Simplified Memory-Bounded A* (SMA*):** A memory-bounded version of A* that discards nodes when memory limits are reached, keeping only the most promising nodes.
 - **How it works:** When the memory limit is exceeded, SMA* prunes the least promising paths to free up space for better candidates.
 - **Memory Usage:** Bounded by a preset limit, making it more efficient than A*.

Advantages:

- **Memory Efficiency:** These algorithms reduce the memory usage compared to traditional A*, making them more scalable for large problems.
- **Optimality:** SMA* retains optimality under memory constraints (if the heuristic is admissible).

Disadvantages:

- **Increased Time Complexity:** May require more iterations or time to find a solution due to the trade-off with memory.
- **Loss of Information:** Discarded nodes may lead to re-exploring the same paths later.

3.5.1 Hill-Climbing Search

Hill-Climbing is a local search algorithm that continuously moves toward the direction of increasing value (i.e., a higher score or fitness), aiming to reach a peak (maximum value). It is a greedy approach with no backtracking.

Properties:

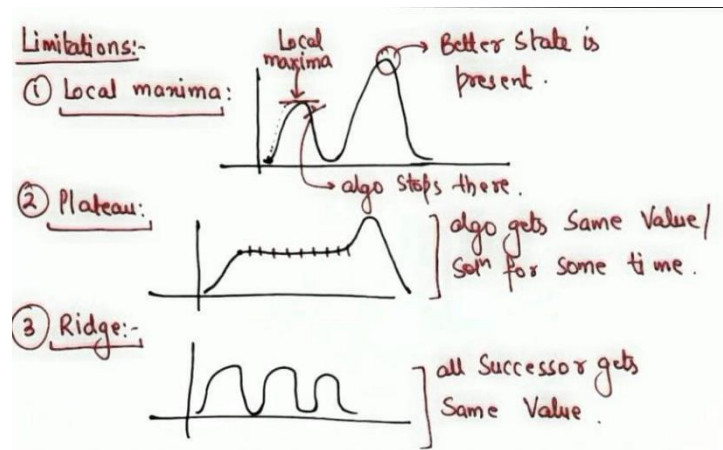
- **Complete:** No, it can get stuck in local optima.
- **Optimal:** No, unless the problem landscape is unimodal (only one peak).
- **Time Complexity:** $O(n)$, where n is the number of neighbours evaluated.

Advantages:

- Simple and easy to implement.
- Efficient for problems where a good initial solution is close to the optimal solution.

Disadvantages:

- **Local Maxima:** A local maximum is a point in the search space where all neighbouring states have a lower value (or higher cost), but it is not the highest point in the entire search space (the global maximum). The search algorithm stops at a local maximum because it appears to be the best solution in the immediate vicinity.



- **Plateau:** A plateau is a flat area of the search space where neighbouring states have the same value. On a plateau, the search may stall, as Hill Climbing relies on detecting an improvement in value to move forward. Without such improvements, the algorithm may stop making progress.

- **Ridges:** are sequence of local maxima that is very difficult for greedy algorithms to navigate. Hill Climbing may "slide off" the ridge and miss the best path. The narrow and difficult-to-navigate nature of ridges makes it challenging for the algorithm to find the right route to the global maximum.

3.5.2 Simulated Annealing Search

Simulated Annealing (SA) is a probabilistic technique for approximating the global optimum of a given function. It mimics the process of annealing in metallurgy, where materials are heated and then cooled to make them reach a state of low energy where they are very strong.



Properties:

- **Complete:** Yes, if given enough time and appropriate cooling.
- **Optimal:** It can approximate the global optimum, but not guaranteed.
- **Time Complexity:** Typically, $O(n)$, but varies with the cooling schedule.

Advantages:

- Can escape local optima by allowing worse solutions at higher temperatures.
- Effective for large search spaces with complex landscapes.

Disadvantages:

- Performance heavily depends on the cooling schedule and parameters.
- Slower convergence compared to other local search algorithms.

3.5.3 Local Beam Search Genetic Algorithm

Local Beam Search is a variant of local search that keeps multiple candidate solutions (beams) rather than one. It can also refer to Genetic Algorithms, which are inspired by natural selection and evolution to find optimal solutions.

Properties:

- **Complete:** Yes, given enough time and generations.
- **Optimal:** Generally, yes, especially with a good selection and mutation strategy, but not guaranteed.
- **Time Complexity:** Typically, $O(ng)$, where n is the number of solutions and g is the number of generations.
- **Space Complexity:** Depends on the number of solutions kept in the population.

Advantages:

- Explores multiple areas of the solution space simultaneously.
- Combines exploration and exploitation through crossover and mutation.

Disadvantages:

- Requires careful tuning of parameters (population size, mutation rates).
- Can converge prematurely on suboptimal solutions without diversity.

3.6 Constraint Satisfaction Problems (CSPs)

A CSP is a problem where the goal is to find a solution that satisfies a set of constraints. **Examples of CSPs:** Sudoku, crossword puzzles, map colouring, and scheduling problems. Each problem is defined by:

- **Variables:** A set of variables $X = \{x_1, x_2, \dots, x_n\}$.
- **Domains:** A set of possible values for each variable $D = \{D_1, D_2, \dots, D_n\}$.
- **Constraints:** A set of constraints that specify which combinations of values are valid for the variables.

Solution Methods:

1. **Backtracking Search:** A depth-first search algorithm for CSPs, which assigns values to variables one at a time and backtracks when a constraint is violated. **Properties:**
 - **Complete:** Yes, but inefficient without improvements.
 - **Optimal:** No, unless all solutions have the same cost.

2. **Constraint Propagation:** Uses techniques like arc-consistency to reduce the domain of variables by enforcing constraints early on, thereby pruning the search space.

Advantages:

- CSPs model many real-world problems naturally.
- Constraint propagation reduces the need for exhaustive search.

Disadvantages: Can be computationally expensive if the problem has many constraints or variables.

3.7 Local Search for CSPs

Local search methods attempt to solve CSPs by starting with an initial assignment of values to variables and iteratively improving the solution by making local changes.

Types of Local Search:

1. **Min-Conflicts Heuristic:** A heuristic that tries to minimize the number of constraint violations (conflicts) by iteratively changing the value of a variable to reduce the number of conflicts.
 - **How it works:** Randomly selects a conflicted variable and assigns it a value that results in the fewest conflicts.
 - **Example:** Solving the n-queens problem, where queens are placed on a chessboard in a way that no two queens threaten each other.
2. **Simulated Annealing for CSPs:** Allows worsening moves with some probability to escape local minima, gradually reducing the probability of these moves.
 - **How it works:** Like simulated annealing for optimization problems, it can escape local optima by making occasional random (worsening) moves, with a probability that decreases over time.

Advantages:

- **Memory Efficiency:** Local search methods often use less memory than backtracking or other complete search methods.
- **Effective for Large Problems:** Particularly useful for CSPs where complete algorithms would be too slow or memory-intensive.

Disadvantages:

- **No Guarantee of Optimality:** Local search may get stuck in local optima.
- **Incomplete:** Local search does not guarantee finding a solution, even if one exists.

3.8 Comparative study

Criterion	Greedy First Search	Best- A* Search	Memory-Bounded Heuristic Search	CSPs	Local Search for CSPs
Search Strategy	Expands node closest to the goal	Combines path cost and heuristic	Memory-efficient variants of A*	Find assignments for variables that satisfy constraints	Starts with an initial solution and improves iteratively

Heuristic Used	$h(n)$ (cost to goal)	$f(n)=g(n)+h(n)$	Bounded versions of $f(n)=g(n)+h(n)$	Heuristics to propagate constraints, reduce search space	Min-conflicts heuristic, simulated annealing
Completeness	No, can get stuck in loops	Yes, with admissible heuristic	Yes (SMA*)	Yes, if using backtracking	No, depends on the problem
Optimality	No, doesn't consider path cost	Yes, with admissible heuristic	Yes (SMA* is optimal under memory limits)	Not guaranteed without additional strategies	No, can get stuck in local optima
Time Complexity	Depends on heuristic quality, generally $O(b^m)$	$O(b^m)$	More efficient than A* in terms of memory	Depends on the constraint propagation and backtracking	Depends on iterations and conflicts
Memory Usage	Moderate to high	High, since all nodes are stored	Bounded by memory constraints	Moderate to high, depending on propagation	Low, only current assignments are stored
Best For	Fast solutions in well-formed heuristic problems	Optimal solution finding with admissible heuristic	Memory-constrained problems	Scheduling, planning, puzzles	Large CSPs where exhaustive search is impractical

4 Adversarial Search

Adversarial search is used in multi-agent environments, typically in competitive settings such as games. Here, agents must consider the actions of other agents, who may have conflicting goals.

4.1 Games

Games in the context of AI refer to a structured problem in which two or more players take turns making moves to maximize their own advantage while minimizing the opponent's advantage. These games can be:

- **Deterministic:** The outcome is completely determined by the players' actions (e.g., chess, checkers).
- **Zero-Sum:** A gain by one player results in an equal loss by the other (e.g., tic-tac-toe).

Examples:

- **Perfect Information Games:** Both players know the full game state (e.g., chess, Go).
- **Imperfect Information Games:** Some information is hidden from the players (e.g., poker).

4.2 Optimal Decisions & Strategies in Games

In adversarial games, an optimal strategy is one that guarantees the best possible outcome for a player, regardless of the opponent's actions. Each player aims to maximize their own payoff while minimizing the opponent's payoff.

Key Concept - Utility Function: Represents the outcome of the game, with values typically ranging from a win (+1), loss (-1), or draw (0).

Strategies:

- **Maximizing Player (MAX):** Seeks to maximize the outcome or utility.
- **Minimizing Player (MIN):** Seeks to minimize the opponent's utility, i.e., maximize their own gain by minimizing the other's.